

Vite & Gourmand

Documentation technique

Réflexions technologiques initiales

Choix de la stack

Le cahier des charges n'imposait aucune technologie précise (hormis l'usage conjoint d'une base relationnelle et d'une base non relationnelle). Le choix s'est porté sur une stack volontairement simple et sans framework :

Couche	Choix	Justification
Front-end	HTML5, CSS, JavaScript vanilla	Aucune dépendance de build (webpack, npm...) à maintenir ; suffisant pour les interactions demandées (filtres AJAX, calcul de prix en direct).
Back-end	PHP natif + PDO	PDO impose les requêtes préparées (protection injections SQL), sans la courbe d'apprentissage d'un framework complet ; adapté à un hébergement mutualisé classique.
Base relationnelle	MySQL / MariaDB	Standard de fait dans l'écosystème PHP (XAMPP), gratuit, bien documenté.
Base non relationnelle	MongoDB	Modèle document adapté au reporting dénormalisé (agrégations rapides pour les statistiques admin), en complément du relationnel qui reste la source de vérité transactionnelle.
Emails	PHP <code>mail()</code> + Mercury Mail (local)	Aucune dépendance externe (pas de compte SMTP tiers requis pour développer/tester), remplaçable par un vrai relais SMTP en production.

Architecture applicative

Architecture en couches simples, sans routeur ni ORM :

- **public/** — unique racine web exposée (sécurité : le code métier n'est jamais accessible directement en HTTP).
- **src/Config** — connexion PDO, chargement des variables d'environnement, bootstrap de session.
- **src/Models** — accès aux données via PDO (requêtes préparées).
- **src/Services** — règles métier (calcul de prix, envoi de mails, authentification, statistiques Mongo).
- **src/Views/partials** — gabarits communs (en-tête, pied de page, navigation).

Sécurité

- Mots de passe hachés avec `password_hash()` (bcrypt), jamais stockés en clair.

- Toutes les requêtes SQL utilisent des requêtes préparées PDO (protection contre les injections SQL).
- Toutes les sorties HTML passent par un échappement systématique (protection XSS).
- Jeton anti-CSRF sur tous les formulaires de modification.
- Contrôle d'accès par rôle sur chaque page sensible (`AuthService::requireRole()`).
- Sessions cookies en `HttpOnly` + `SameSite=Lax` .
- Jetons de réinitialisation de mot de passe stockés hachés (SHA-256) et à durée de vie limitée (1h).

Configuration de l'environnement de travail

1. XAMPP (Apache, PHP 8.1+, MySQL/MariaDB) pour l'exécution locale.
2. Un VirtualHost Apache dédié pointant vers `public/` , pour que les chemins racine-relatifs (`/assets/...`) fonctionnent correctement (voir README.md).
3. MongoDB Community Server + `mongosh` pour la base non relationnelle.
4. Git, avec le modèle de branches décrit dans le README (main / développement / feature).
5. Un fichier `.env` (non versionné) dérivé de `.env.example` pour les identifiants de connexion.

Le détail pas-à-pas de l'installation figure dans le fichier `README.md` à la racine du dépôt.

Modèle conceptuel de données

Schéma relationnel complet (voir aussi `database/schema.sql`). Les tables de liaison (`menu_regime` , `menu_plat` , `plat_allergene`) sont représentées comme des relations many-to-many.

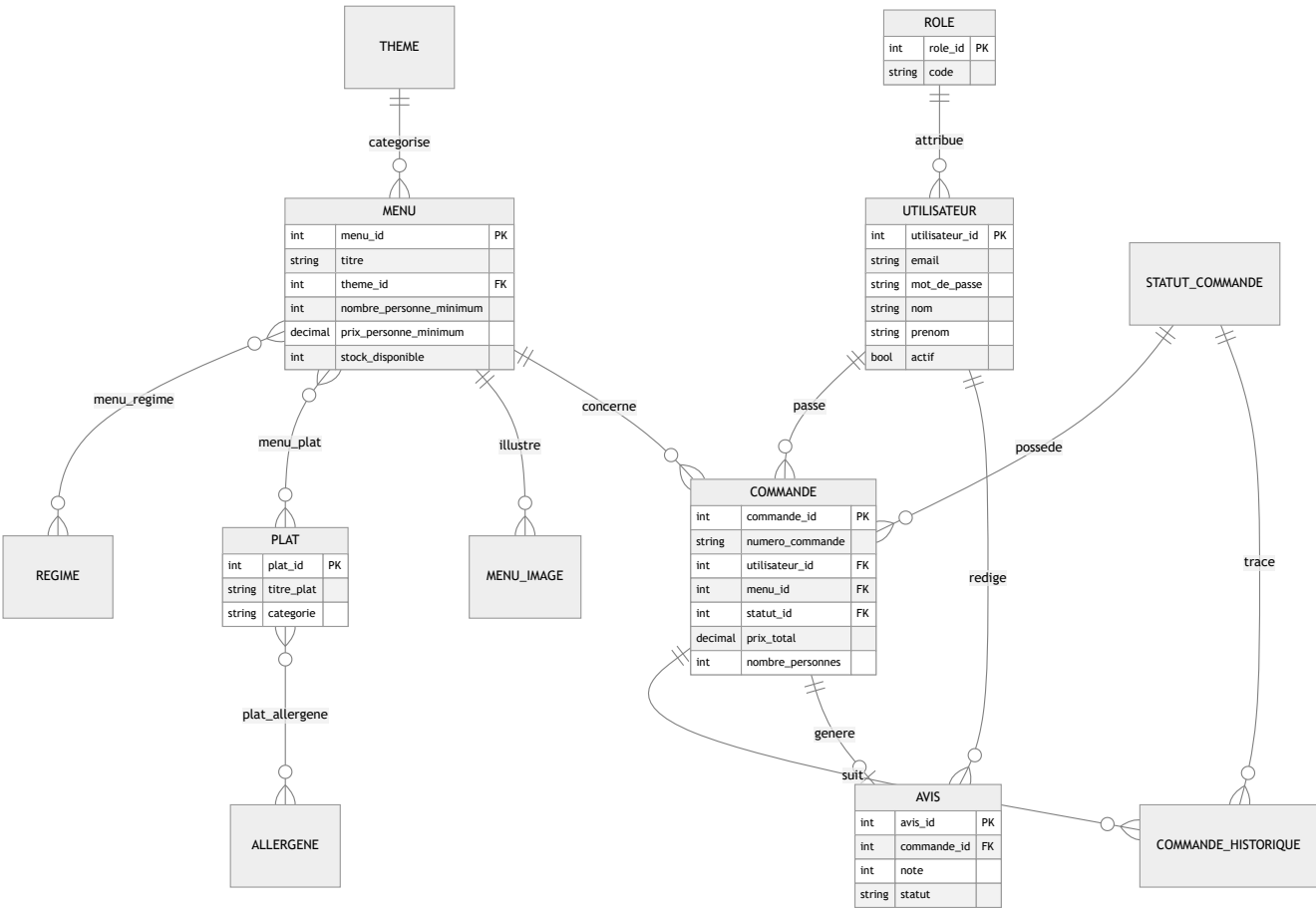


Diagramme de cas d'utilisation

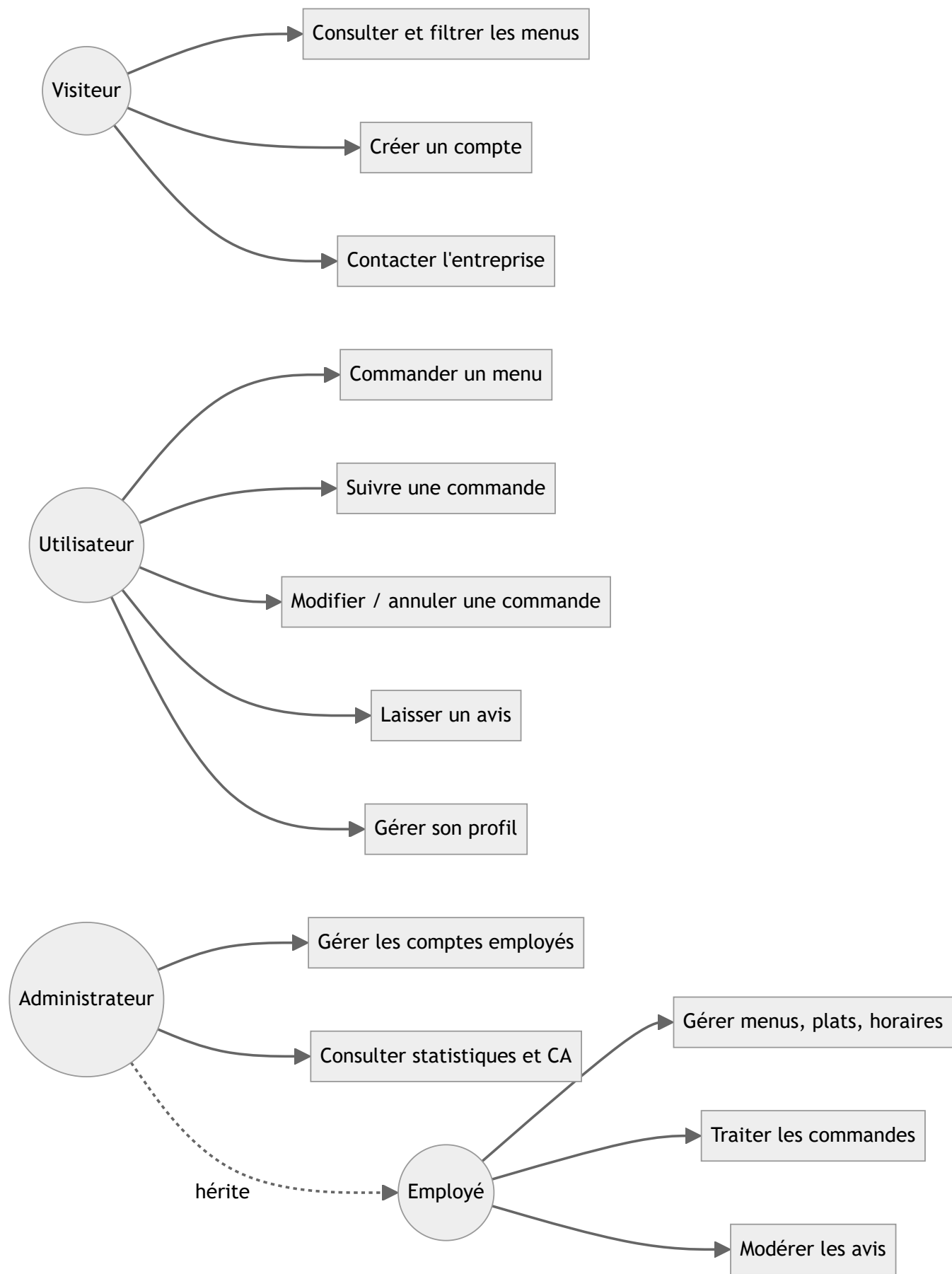
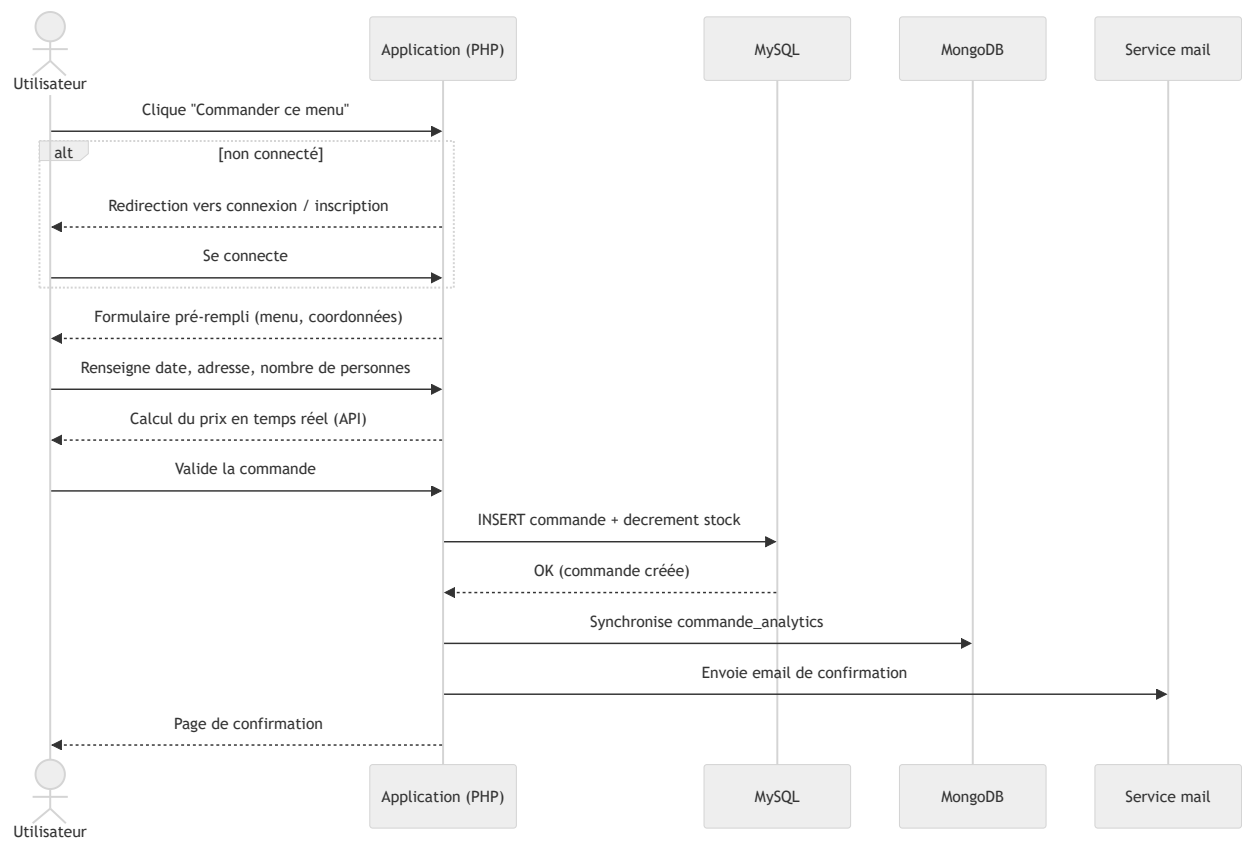


Diagramme de séquence — Passer une commande



Documentation du déploiement

Démarche

Le déploiement suit le même modèle que l'installation locale, en remplaçant les services locaux (XAMPP, Mercury Mail) par leurs équivalents managés sur la plateforme choisie :

1. Provisionner une base MySQL/MariaDB managée et importer `database/schema.sql` puis `database/seed.sql` (avec `--default-character-set=utf8mb4`).
2. Provisionner une base MongoDB managée (ou Atlas) et exécuter `database/mongodb/init.js` .
3. Configurer les variables d'environnement de production (voir `.env.example`) : identifiants base de données, URI MongoDB, configuration SMTP réelle.
4. Définir `public/` comme racine web du service de déploiement.
5. Déployer le code depuis la branche `main` du dépôt Git.
6. Vérifier les parcours critiques (inscription, commande, changement de statut, emails) en production avant la mise à disposition finale.

La plateforme de déploiement définitive (fly.io, Heroku, Azure, Vercel...) sera précisée et documentée en détail (étapes de configuration spécifiques, URL de production) une fois choisie ; la démarche ci-dessus reste valable quelle que soit la plateforme retenue, ces dernières fonctionnant toutes sur ce même principe (build + variables d'environnement + base de données managée).

